First Passage Percolation: Basic Definitions in Lean 4

Generated from FPP_Basics.lean

January 30, 2026

1 Overview

This document presents the formalization of First Passage Percolation (FPP) basics in Lean 4 with Mathlib.

2 Source Code

Listing 1: FPP_Basics.lean

```
import Mathlib
import percolation.PercolationZd

open scoped BigOperators
open scoped Topology

namespace FPP

noncomputable section

variable {d : ℕ}

/-- The vertex set `Z^d`, represented as functions `Fin d → ℤ`. -/
abbrev Zd (d : ℕ) : Type := Fin d → ℤ

/-- The ambient space `R^d`, represented as functions `Fin d → ℝ`. -/
abbrev Rd (d : ℕ) : Type := Fin d → ℝ

/-- Coordinatewise floor map `R^d → Z^d`. -/
def floorZd (x : Rd d) : Zd d := fun i => Int.floor (x i)

/-- Nearest-neighbor relation on `Z^d`. -/
def IsNN (x y : Zd d) : Prop :=
  ∃ i : Fin d, (∀ j : Fin d, j ≠ i → x j = y j) ∧ (y i = x i + 1 ∨ y i
      = x i - 1)

/-- Edge weights on oriented pairs of vertices (no symmetry assumed).
    -/
abbrev Weights (d : ℕ) : Type := Zd d → Zd d → ℝ≥0∞

/-- A self-avoiding nearest-neighbor path in `Z^d`, encoded as a list
    of vertices.
```

```
30
31  ‘adj‘ says successive vertices are nearest-neighbors.
32  ‘nodup‘ says the vertex list is self-avoiding.
33  -/
34  structure SAPath (d : ℕ) where
35    verts : List (Zd d)
36    nonempty : verts ≠ []
37    adj : verts.Chain' (IsNN (d := d))
38    nodup : verts.Nodup
39
40  namespace SAPath
41
42  variable {d : ℕ}
43
44  /-- Start vertex of a path. -/
45  def start (γ : SAPath d) : Zd d := γ.verts.head!
46
47  /-- End vertex of a path. -/
48  def finish (γ : SAPath d) : Zd d := γ.verts.getLast γ.nonempty
49
50  /-- The oriented edge list of consecutive vertex pairs. -/
51  def edges (γ : SAPath d) : List (Zd d × Zd d) :=
52    γ.verts.zip γ.verts.tail
53
54  /-- The number of edges in the path. -/
55  def edgeLength (γ : SAPath d) : ℕ := γ.edges.length
56
57  /-- Passage time of a path for a given weight function. -/
58  def time (w : Weights d) (γ : SAPath d) : ℝ≥0∞ :=
59    (γ.edges.map (fun e => w e.1 e.2)).sum
60
61  /-- The set of self-avoiding paths from ‘x‘ to ‘y‘. -/
62  def Between (x y : Zd d) : Set (SAPath d) :=
63    {γ | γ.start = x ∧ γ.finish = y}
64
65  end SAPath
66
67  /-- First-passage time on ‘Z^d‘: infimum of passage times over self-
        avoiding paths. -/
68  def passageTimeZd (w : Weights d) (x y : Zd d) : ℝ≥0∞ :=
69    sInf (Set.image (SAPath.time (d := d) w) (SAPath.Between (d := d) x y
        ))
70
71  /-- First-passage time on ‘R^d × R^d‘ defined by flooring coordinates.
        -/
72  def passageTimeRd (w : Weights d) (x y : Rd d) : ℝ≥0∞ :=
73    passageTimeZd (d := d) w (floorZd (d := d) x) (floorZd (d := d) y)
74
75  /-- Geodesics in ‘Z^d‘: self-avoiding paths from ‘x‘ to ‘y‘ attaining
        the passage time. -/
76  def GeodesicsZd (w : Weights d) (x y : Zd d) : Set (SAPath d) :=
77    {γ | γ ∈ SAPath.Between (d := d) x y ∧ SAPath.time (d := d) w γ =
          passageTimeZd (d := d) w x y}
78
```

```
/-- Geodesics in `R^d`, defined by flooring the endpoints. -/
def GeodesicsRd (w : Weights d) (x y : Rd d) : Set (SAPath d) :=
  GeodesicsZd (d := d) w (floorZd (d := d) x) (floorZd (d := d) y)

/-- Subadditivity of passage times on `Z^d`. -/
theorem passageTimeZd_subadditive (w : Weights d) (x y z : Zd d) :
    passageTimeZd (d := d) w x z ≤ passageTimeZd (d := d) w x y +
      passageTimeZd (d := d) w y z := by
  sorry

/-- Subadditivity of passage times on `R^d` (via flooring). -/
theorem passageTimeRd_subadditive (w : Weights d) (x y z : Rd d) :
    passageTimeRd (d := d) w x z ≤ passageTimeRd (d := d) w x y +
      passageTimeRd (d := d) w y z := by
  sorry

section Random

open MeasureTheory
open scoped BigOperators

variable {Ω : Type*} [MeasurableSpace Ω]
variable (ℙ : Measure Ω) [ProbabilityTheory.IsProbabilityMeasure ℙ]

/-- A random environment: `ω ↦ weight function` on oriented edges. -/
abbrev Env (d : ℕ) (Ω : Type*) : Type := Ω → Weights d

variable (τ : Env d Ω)

/-- Placeholder for the usual i.i.d. assumptions on edge weights. -/
def IIDWeights : Prop := True

/-- Placeholder for stationarity under lattice translations. -/
def Stationary : Prop := True

/-- Placeholder for ergodicity under lattice translations. -/
def Ergodic : Prop := True

/-- The standard basis vector `e_i` in `Z^d`. -/
def stdBasis (i : Fin d) : Zd d := fun j => if j = i then (1 : ℤ) else
    0

/-- The weight of the oriented edge from `0` to `ś e_i`.

We index the sign by `Bool`: `true` means `+e_i`, `false` means `-e_i`.
-/
def neighborWeight (w : Weights d) (i : Fin d) (sgn : Bool) : ℝ≥0∞ :=
  if sgn then w 0 (stdBasis (d := d) i) else w 0 (-stdBasis (d := d) i)

/-- The minimum of the `2d` weights adjacent to the origin.

This is written as `sInf` of the range so it is defined for all `d`.
When `d ≥ 1`, this agrees with the minimum over the `2d` neighbors.
-/
```

```
def minNeighborWeight (w : Weights d) : ℝ≥0∞ :=
  sInf (Set.range (fun p : Fin d × Bool => neighborWeight (d := d) w p
     .1 p.2))

/-- The integrability hypothesis `E[min_{i=1..2d} τ_i] < ∞`.

We encode it as finiteness of the `lintegral` of the minimum adjacent
    weight.
-/
def minNeighborIntegrable : Prop :=
  (∫⁻ ω, minNeighborWeight (d := d) (τ ω) ∂ℙ) < ⊤

/-- Random first-passage time on `Z^d`. -/
def Tzd (x y : Zd d) : Ω → ℝ≥0∞ := fun ω => passageTimeZd (d := d) (
    τ ω) x y

/-- Random first-passage time on `R^d` (via flooring). -/
def Trd (x y : Rd d) : Ω → ℝ≥0∞ := fun ω => passageTimeRd (d := d) (
    τ ω) x y

/-- Existence of a time constant under an integrability hypothesis.

This is a typical Kingman subadditive ergodic theorem conclusion.
The statement is given for the `R^d` version that uses flooring.
-/
theorem timeConstant_exists
    (hd : 1 ≤ d)
    (hIID : IIDWeights (d := d) (ℙ := ℙ) τ)
    (hStat : Stationary (d := d) (ℙ := ℙ) τ)
    (hErg : Ergodic (d := d) (ℙ := ℙ) τ)
    (hmin : minNeighborIntegrable (d := d) (ℙ := ℙ) τ) :
    ∃ μ : Rd d → ℝ, ∀ x : Rd d,
      (∀ᵐ ω ∂ℙ, Tendsto (fun n : ℕ =>
          (Trd (d := d) (ℙ := ℙ) τ 0 (n • x) ω).toReal / (n : ℝ))
        Filter.atTop (𝓝 (μ x))) := by
  sorry

/-- Event: there exists a self-avoiding path starting at `0` of edge-
    length `n`
with passage time strictly smaller than `c n`. -/
def FastPathEvent (n : ℕ) (c : ℝ) : Set Ω :=
  {ω | ∃ γ : SAPath d,
      γ.start = 0 ∧ γ.edgeLength = n ∧ SAPath.time (d := d) (τ ω) γ <
          ENNReal.ofReal (c * n)}

/-- The percolation parameter `p0 := P( τ(0,e₁) = 0 )` as a real number
    .

This uses a fixed coordinate `i0` built from `hd : 1 ≤ d`.
-/
def pZero (hd : 1 ≤ d) : ℝ :=
  let i0 : Fin d := ⟨0, Nat.lt_of_lt_of_le Nat.zero_lt_one hd⟩
  (ℙ {ω | (τ ω) 0 (stdBasis (d := d) i0) = 0}).toReal
```

```
/-- Exponential bound on the probability of very fast self-avoiding paths,
under the subcritical percolation condition `pZero < pcd d`.

The constant `pcd d` is assumed to be defined in `percolation/PercolationZd.lean`.
-/
theorem prob_fastPath_lt_exp
    (hd : 1 ≤ d)
    (hIID : IIDWeights (d := d) (ℙ := ℙ) τ)
    (hsub : pZero (d := d) (ℙ := ℙ) τ hd < pcd d) :
    ∃ c : ℝ, 0 < c ∧ ∀ n : ℕ,
      ℙ (FastPathEvent (d := d) (ℙ := ℙ) τ n c) < ENNReal.ofReal (Real.exp (-c * n)) := by
  sorry

/-- Existence of geodesics as a consequence of an exponential fast-path bound.

The conclusion states that almost surely, for every pair of lattice points `x y`,
there is at least one geodesic from `x` to `y`.
-/
theorem geodesic_exists_of_fastPath_bound
    (hd : 1 ≤ d)
    (hIID : IIDWeights (d := d) (ℙ := ℙ) τ)
    (hbound : ∃ c : ℝ, 0 < c ∧ ∀ n : ℕ,
      ℙ (FastPathEvent (d := d) (ℙ := ℙ) τ n c) < ENNReal.ofReal (Real.exp (-c * n))) :
    ∀ᵐ ω ∂ℙ, ∀ x y : Zd d, (GeodesicsZd (d := d) (τ ω) x y).Nonempty := by
  sorry

end Random

end FPP
```